EST-744

Embeddings & Representation Learning

Juan Zamora O.

Deep Learning





The Problem: Text is Symbolic

How do we feed words to neural networks?

Traditional approaches (don't work well):

- **One-hot encoding**: "dog" → [0, 0, ..., 1, 0, ..., 0] (10,000 dimensions for 10k vocab)
  - No meaning captured (all words equally different)
  - Huge, sparse vectors

Better idea: Learn dense representations where meaning is encoded

From Symbols to Vectors

Embedding: A learned mapping from words to dense vectors (typically 50-300 dimensions)

```
"dog"   → [0.23, -0.15, 0.08, ..., 0.42]
"cat"   → [0.25, -0.13, 0.07, ..., 0.41]
"king"  → [0.12,  0.45, -0.22, ..., 0.61]
"queen" → [0.10,  0.50, -0.20, ..., 0.58]
```

Key insight: Similar words have similar vectors

Distributional Hypothesis

“You shall know a word by the company it keeps” (Firth, 1957)

- Words appearing in similar contexts have similar meanings
- Example: “dog” and “cat” both appear with “pet”, “cute”, “furry”
- So their vectors should be close!

This principle powers modern embeddings

Skip-Gram Model

Idea: Predict context words from a target word

```
Context: "I love my [dog] very much"
Target: "dog"
Predict: "love", "my", "dog", "very", "much"
```

- Train a small network: input word → hidden layer → predict context words
- Throw away the prediction layer
- Keep the hidden layer weights → **embedding vectors**

Training Process

1. Extract (target, context) word pairs from corpus
2. Feed target word one-hot encoding
3. Network learns to predict context words
4. Emergent property: similar words → similar vectors
5. After training, word vectors capture semantic meaning

Semantic Relationships

$$king - man + woman \approx queen$$

Why? - “king” has male attributes - “man” encodes maleness - Subtracting it and adding “woman” adjusts gender - Yields “queen”

Other analogies: - France - Paris + London ≈ England (capital cities) - good - bad ≈ positive - negative (sentiment)

Pre-trained Embeddings

You don't train embeddings from scratch; use pre-trained ones:

Embedding	Dimension	Corpus	Notes
Word2Vec	300	Google News	Fast, intuitive
GloVe	300	Wikipedia, Gigaword	Balances local & global context
FastText	300	Common Crawl	Handles OOV, subword info

For this course: Use `gensim` (Word2Vec) or `spacy` (GloVe)

Using Embeddings in Models

Instead of one-hot encoding, use an **embedding layer**:

```
keras.layers.Embedding(
    input_dim=vocab_size,     # How many words?
    output_dim=embedding_dim, # Vector dimension (e.g., 128)
    input_length=max_length   # Sequence length
)
```

Workflow: 1. Tokenize text: “dog cat” → [2, 5] 2. Embedding layer maps: [2, 5] → [[0.23, -0.15, ...], [0.19, 0.34, ...]] 3. Feed to downstream layers (LSTM, CNN, etc.)

Option 1: Learn embeddings from scratch (small data)

Option 2: Load pre-trained, fine-tune (recommended)

From Words to Documents

How do we represent an entire sentence or document?

Approach 1: Average Embedding

```
sentence = "dog and cat"
embedding = (embed["dog"] + embed["cat"]) / 2
```

Simple, fast, surprisingly effective!

Approach 2: TF-IDF + Embeddings - Weight embeddings by importance (TF-IDF scores) - Gives more influence to meaningful words

Approach 3: Learned Pooling - Let the network learn how to combine word vectors

Task: Sentiment Classification

Goal: Predict if a movie review is positive or negative

Data: IMDb dataset (25,000 reviews)

Approach: 1. Load pre-trained Word2Vec embeddings 2. Tokenize reviews, convert to sequences 3. Average embeddings to get document vectors 4. Feed to simple classifier

Loading & Preparing Embeddings

```python
from gensim.models import KeyedVectors
import numpy as np

# Load pre-trained Word2Vec
w2v = KeyedVectors.load_word2vec_format(
    'GoogleNews-vectors-negative300.bin', binary=True
)

# Create embedding matrix for our vocabulary
vocab_size = 10000
embedding_dim = 300
embedding_matrix = np.random.randn(vocab_size, embedding_dim)

for word_idx, word in enumerate(word_list[:vocab_size]):
    if word in w2v:
        embedding_matrix[word_idx] = w2v[word]
```

Classification with Averaged Embeddings

```
import tensorflow as tf
from tensorflow import keras

# Load IMDb dataset
(x_train, y_train), (x_test, y_test) = \
    keras.datasets.imdb.load_data(num_words=10000)

# Convert token sequences to embedding sequences
x_train_emb = np.array([
    embedding_matrix[seq].mean(axis=0) for seq in x_train
])
x_test_emb = np.array([
    embedding_matrix[seq].mean(axis=0) for seq in x_test
])

# Simple classifier on averaged embeddings
model = keras.Sequential([
```

t-SNE: Visualizing High-Dimensional Embeddings

```python
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt

# Reduce embeddings to 2D for visualization
tsne = TSNE(n_components=2, random_state=42)
embeddings_2d = tsne.fit_transform(embedding_matrix[:1000])

# Plot
plt.scatter(embeddings_2d[:, 0], embeddings_2d[:, 1], alpha=0.5)
for i, word in enumerate(word_list[:100]):
    plt.annotate(word,
                 (embeddings_2d[i, 0], embeddings_2d[i, 1]),
                 fontsize=8)
plt.title('Word Embeddings Visualization')
plt.show()
```

What to look for: Related words cluster together

Testing Embedding Quality

```python
def word_analogy(w1, w2, w3, model, top_n=5):
    """Solve: w1 is to w2 as w3 is to ?"""
    # king - man + woman = ?
    result = model.most_similar(
        positive=[w2, w3],
        negative=[w1],
        topn=top_n
    )
    return result

# Test
print(word_analogy("king", "man", "woman", w2v))
# Expected: queen, princess, etc.

print(word_analogy("Paris", "France", "London", w2v))
# Expected: England, Britain, etc.
```

Results

```
test_loss, test_acc = model.evaluate(x_test_emb, y_test)
print(f"Test accuracy: {test_acc:.4f}")

# View predictions
predictions = model.predict(x_test_emb[:10])
for pred, truth in zip(predictions[:10], y_test[:10]):
    print(f"Predicted: {pred[0]:.2f}, True: {truth}")
```

Expected: ~85-90% accuracy with averaged embeddings

Why not perfect? - Averaging loses word order (“not good” = “good not”) - Loses context-specific meanings (ambiguity) - Solution: Use sequential models (Class 3)

Embeddings Recap

- Words as dense vectors (50-300 dimensions)
- Similarity in vector space = semantic similarity
- Pre-trained embeddings (Word2Vec, GloVe, FastText)
- Using embeddings in Keras models
- Document representation (averaging, TF-IDF)
- Visualization and evaluation

Why Embeddings Matter

- **Compact**: 300 dims vs. 10,000+ one-hot
- **Meaningful**: capture semantic relationships
- **Transferable**: use across tasks
- **Fast**: pre-trained, no need to retrain from scratch

Troubleshooting

Problem	Cause	Solution
OOV words	Pre-trained doesn't know rare words	Use FastText or subword embeddings
Low accuracy	Averaging loses word order	Use RNN/CNN (next class)
Slow inference	Huge embedding files	Use smaller pre-trained (e.g., 50-100 dims)
Overfitting	Small dataset	Use pre-trained, freeze embeddings

Hands-On Tasks

1. **Load different embeddings**: Compare Word2Vec vs. GloVe results
2. **Embedding dimensions**: Train with 50, 100, 300 dims. Does larger help?
3. **Frozen vs. trainable**: Train with `trainable=False` vs. `True`. Which generalizes better?
4. **Analogies**: Test more word analogies. Find ones that fail.
5. **Visualization**: t-SNE visualize 2,000 word embeddings. Describe patterns.
6. **Modify averaging**: Instead of mean, try max pooling or weighted pooling. Results?

Limitations of Averaging

Averaging embeddings ignores: - **Word order**: “good movie” vs. “movie good” - **Context**: Same word, different meanings - **Long-range dependencies**: Connections between distant words

Class 3: Text Classification with RNNs/LSTMs - Sequential models that process words in order - Remember context across sequences - Much better accuracy on real tasks

Questions & Discussion

Thank you!

Key takeaway: Embeddings transform symbolic text into meaningful numeric vectors. This is the bridge between language and deep learning.

Next: How do we capture word order and sequence structure?